Problem 6: Chunked DeltaNet and the WY representation

GStar Bootcamp

Abstract

Derive the triangular correction system, implement its inverse and WY factors, and reuse linear output kernels.

Starter file: `problem_6.py`. **Checker:** `python autograder.py -p6`. **Reading path:** Overview, P3, P5. Next: P7.

Mathematical background: a product of corrections becomes a chunk system

Write the P5 update as an affine map on entry memory:

$$\mathbf{S}_t = \mathbf{T}_t\mathbf{S}_{t-1} + \mathbf{B}_t, \quad \mathbf{T}_t = \mathbf{I} - \beta_t \boldsymbol{k}_t\boldsymbol{k}_t^\top, \quad \mathbf{B}_t = \beta_t \boldsymbol{k}_t\boldsymbol{v}_t^\top.$$

After several tokens, the entry memory is multiplied by $\mathbf{T}_C \cdots \mathbf{T}_1$. These matrices generally do not commute, so adding updates computed independently from the same entry state would change the sequential rule.

The alternative is to solve for the actual correction writes. For two local tokens, the first correction depends on $\mathbf{S}_{\text{in}}$; the second must additionally account for the first:

$$\boldsymbol{p}_1 = \beta_1(\boldsymbol{v}_1 - \mathbf{S}_{\text{in}}^\top \boldsymbol{k}_1), \qquad \boldsymbol{p}_2 = \beta_2\big(\boldsymbol{v}_2 - \mathbf{S}_{\text{in}}^\top \boldsymbol{k}_2 - (\boldsymbol{k}_2^\top \boldsymbol{k}_1)\boldsymbol{p}_1\big).$$

Here $\boldsymbol{p}_r$ denotes the local correction whose transpose is row r of $\mathbf{P}$. For a longer chunk, each row depends only on earlier rows. Moving those dependencies to the left gives the unit lower-triangular system below. Solving it exactly preserves P5's update order, rather than approximating it by independent updates [3].

The compact factors then expose an affine chunk map:

$$\mathbf{S}_{\text{out}} = (\mathbf{I} - \mathbf{K}^\top\mathbf{W})\mathbf{S}_{\text{in}} + \mathbf{K}^\top\mathbf{U}.$$

This is the bridge between the product-of-transitions view and reusable matrix-product kernels. The local factors can be prepared in parallel across chunks; the actual corrections $\mathbf{P} = \mathbf{U} - \mathbf{W}\mathbf{S}_{\text{in}}$ still require each entry state. Yang's talk and the accompanying chunkwise derivation give complementary views of this compression [1, 2].

Before coding. Explain why $\mathbf{U}$ alone is sufficient only when the entry-state prediction $\mathbf{W}\mathbf{S}_{\text{in}}$ vanishes, and why causal correction dependencies require a strict triangular mask.

Why a triangular system appears

Within a chunk, stack the actual delta corrections $\boldsymbol{u}_r^\top$ into rows of a matrix $\mathbf{P} \in \mathbb{R}^{C\times D_v}$. Expanding the state before local token r gives

$$\mathbf{S}_{r-1} = \mathbf{S}_{\text{in}} + \sum_{u<r} \boldsymbol{k}_u\boldsymbol{u}_u^\top.$$

Substitute this expression into the residual definition:

$$\boldsymbol{u}_r = \beta_r \left(\boldsymbol{v}_r - \mathbf{S}_{\text{in}}^\top \boldsymbol{k}_r - \sum_{u<r} (\boldsymbol{k}_r^\top \boldsymbol{k}_u) \boldsymbol{u}_u \right). \tag{6.1}$$

Only earlier residuals appear. Define $\mathbf{D}_\beta = \mathrm{Diag}(\beta_1, \ldots, \beta_C)$ and

$$\mathbf{L} = \mathrm{stricttril}(\mathbf{D}_\beta \mathbf{K}\mathbf{K}^\top). \tag{6.2}$$

Then Eq. (6.1) becomes

$$(\mathbf{I} + \mathbf{L})\mathbf{P} = \mathbf{D}_\beta(\mathbf{V} - \mathbf{K}\mathbf{S}_{\text{in}}). \tag{6.3}$$

Since $\mathbf{L}$ is strictly lower triangular, $\mathbf{I} + \mathbf{L}$ has unit diagonal and is invertible. Set

$$\mathbf{A} = (\mathbf{I} + \mathbf{L})^{-1}, \tag{6.4}$$

$$\mathbf{W} = \mathbf{A}\mathbf{D}_\beta \mathbf{K}, \qquad \mathbf{U} = \mathbf{A}\mathbf{D}_\beta \mathbf{V}, \tag{6.5}$$

$$\mathbf{P} = \mathbf{U} - \mathbf{W}\mathbf{S}_{\text{in}}. \tag{6.6}$$

The compact factors $\mathbf{W}, \mathbf{U}$ summarize the chunk's sequence of corrections. This is the WY-style representation used for efficient DeltaNet chunking [3].

Recover the linear-attention structure

Once $\mathbf{P}$ is known, the state and output follow familiar equations:

$$\mathbf{S}_{\text{out}} = \mathbf{S}_{\text{in}} + \mathbf{K}^\top \mathbf{P}, \tag{6.7}$$

$$\mathbf{O} = \left[\mathbf{Q}\mathbf{S}_{\text{in}} + \mathrm{tril}(\mathbf{Q}\mathbf{K}^\top)\mathbf{P} \right]. \tag{6.8}$$

The new work is obtaining the pseudo-values; the final chunk operates like linear attention with corrected values.

Problem 6: triangular inverse step by step

Forward substitution

Let $\boldsymbol{a}_r^\top$ be row r of $\mathbf{A}$ and $\boldsymbol{e}_r^\top$ row r of the identity. The row equation for $(\mathbf{I} + \mathbf{L})\mathbf{A} = \mathbf{I}$ is

$$\boldsymbol{a}_r^\top = \boldsymbol{e}_r^\top - \sum_{u<r} L_{ru} \boldsymbol{a}_u^\top. \tag{6.9}$$

Start at row 1 and build successive rows. The diagonal is one; the upper triangle is zero. Do not use a generic matrix inverse on the host in the submitted Triton primitive.

Blocked strategy

A practical alternative computes small diagonal block inverses in parallel, then merges them. For a block lower-triangular matrix,

$$\begin{bmatrix} \mathbf{A}_0 & 0 \\ \mathbf{C}_0 & \mathbf{B}_0 \end{bmatrix}^{-1} = \begin{bmatrix} \mathbf{A}_0^{-1} & 0 \\ -\mathbf{B}_0^{-1}\mathbf{C}_0\mathbf{A}_0^{-1} & \mathbf{B}_0^{-1} \end{bmatrix}. \tag{6.10}$$

Use Eq. (6.9) inside 16×16 blocks. Merge to 32×32, then 64×64. The supplied control-flow skeleton handles block selection; you complete the correction arithmetic.

A scalar-key example

For three tokens with $k_r = 1$ and $\beta_r = 1$,

$$\mathbf{L} = \begin{bmatrix} 0 & 0 & 0 \\ 1 & 0 & 0 \\ 1 & 1 & 0 \end{bmatrix}, \quad \mathbf{A} = \begin{bmatrix} 1 & 0 & 0 \\ -1 & 1 & 0 \\ 0 & -1 & 1 \end{bmatrix}.$$

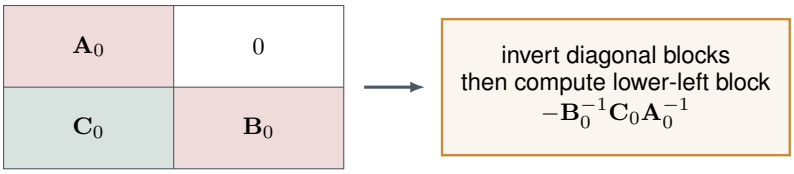


Figure 1. A triangular inverse can be assembled from smaller triangular inverses and matrix products. The upper-right block remains zero.

With zero entry state, $\mathbf{U} = (v_1, v_2 - v_1, v_3 - v_2)^\top$. Accumulating these corrections makes the state equal the most recent value, as the recurrent delta rule predicts.

Problem 6: implementation steps and checkpoints

1. **Strict matrix and triangular primitive (50 points).** Accumulate $\mathbf{K}\mathbf{K}^\top$ over key tiles, multiply each row by that row's β, and zero the diagonal and upper triangle. Implement the inverse using the provided substitution/merge outline.
2. **WY products (35 points).** Form $\mathbf{W}$ and $\mathbf{U}$ using Eq. (6.5). Store their documented shapes. Scaling by β acts on the token row, not the key/value column.
3. **Pseudo-values and states (35 points).** For each chunk, compute $\mathbf{P} = \mathbf{U} - \mathbf{W}\mathbf{S}_{\text{in}}$, store it, and advance the state using Eq. (6.7).
4. **Output kernel (30 points).** Reuse your completed Problem 3 output kernel with pseudo-values in place of values. The P6 file already imports that kernel; keep its inclusive output mask.

```
# _chunk_wy_representation
# TODO P6.A: strict lower matrix, with row-wise beta weighting.
# TODO P6.B: A = inverse of the unit lower-triangular system.
# TODO P6.C: produce W [C, Dk] and U [C, Dv].

# _chunk_state_forward
# TODO P6.D: P = U - W @ entry_state.
# TODO P6.E: advance state with K.T @ P.

# _chunk_output_forward
# Reuse completed P3 output: entry-state read + inclusive scores @ P.
```

Checkpoint table

Table 1. Checkpoint table.

Intermediate	Independent check
$\mathbf{L}$	Strictly lower; compare with a small PyTorch product
$\mathbf{A}$	$(\mathbf{I} + \mathbf{L})\mathbf{A}$ approximates identity
$\mathbf{W}, \mathbf{U}$	Compare with a PyTorch triangular solve
$\mathbf{P}$	Compare each row with sequential residuals inside one chunk
$\mathbf{S}_{\text{out}}, \mathbf{O}$	Compare with the Problem 5 recurrence

Two important distinctions

The WY matrix is strictly lower because the residual at row r depends on earlier corrections. The output matrix is inclusive because the current correction has already been written. Query scale belongs in the output computation; it must not enter $\mathbf{L}$, $\mathbf{A}$, or the memory update.

Check correlated keys, $\beta = 0$, $\beta = 1$, nonzero initial states, and partial last chunks. Debug $\mathbf{A}$ before debugging the full attention output.

Problem 6: solving a two-token correction system by hand

Take $D_k = D_v = 1$, $C = 2$, $k_1 = k_2 = 1$, $\beta_1 = 1/2$, $\beta_2 = 1$, values $(v_1, v_2) = (2, 5)$ and entry state $S_{\text{in}} = 1$. Sequential execution gives

$$u_1 = \tfrac{1}{2}(2-1) = \tfrac{1}{2}, \quad S_1 = \tfrac{3}{2}, \qquad u_2 = 5 - \tfrac{3}{2} = \tfrac{7}{2}, \quad S_2 = 5.$$

The chunk system uses

$$\mathbf{D}_\beta = \begin{bmatrix} 1/2 & 0 \\ 0 & 1 \end{bmatrix}, \quad \mathbf{L} = \begin{bmatrix} 0 & 0 \\ 1 & 0 \end{bmatrix}, \quad \mathbf{A} = \begin{bmatrix} 1 & 0 \\ -1 & 1 \end{bmatrix}.$$

Its factors are

$$\mathbf{W} = \begin{bmatrix} 1/2 \\ 1/2 \end{bmatrix}, \qquad \mathbf{U} = \begin{bmatrix} 1 \\ 4 \end{bmatrix}, \qquad \mathbf{P} = \mathbf{U} - \mathbf{W}S_{\text{in}} = \begin{bmatrix} 1/2 \\ 7/2 \end{bmatrix}.$$

These pseudo-values are exactly the two sequential correction writes. The state update adds their sum to the entry state, and inclusive local outputs recover S_1 and S_2 for unit queries.

Why beta weights rows

The equation for correction r multiplies its entire residual by β_r:

$$P_{ra} + \sum_{j<r} \beta_r (\boldsymbol{k}_r^\top \boldsymbol{k}_j) P_{ja} = \beta_r \big(V_{ra} - (\mathbf{K}\mathbf{S}_{\text{in}})_{ra}\big).$$

Thus the coefficient of an earlier correction j uses β_r, not β_j. Multiplication by $\mathbf{D}_\beta$ acts on rows. A constant-beta test cannot detect row/column confusion; a varying-beta case such as the example above can.

The inverse exists without a conditioning assumption

A strictly lower $C \times C$ matrix satisfies $\mathbf{L}^C = \mathbf{0}$. Consequently,

$$(\mathbf{I} + \mathbf{L})^{-1} = \mathbf{I} - \mathbf{L} + \mathbf{L}^2 - \cdots + (-1)^{C-1}\mathbf{L}^{C-1}.$$

This finite series proves algebraic invertibility. It does not guarantee small floating-point errors for every key sequence. The assignment uses substitution and blocked products rather than explicitly evaluating this series; validate the residual $(\mathbf{I} + \mathbf{L})\mathbf{A} - \mathbf{I}$ before using the inverse downstream.

Problem 6: the roles of W, U, and pseudo-values

Chunk-local preprocessing versus state-dependent work

The inverse $\mathbf{A}$, entry factor $\mathbf{W}$, and base values $\mathbf{U}$ depend only on this chunk's keys, values, and beta. They can be built independently for every chunk. Actual pseudo-values $\mathbf{P} = \mathbf{U} - \mathbf{W}\mathbf{S}_{\text{in}}$ depend on the entry state and therefore belong to the sequential boundary pass.

Table 2. Chunk-local preprocessing versus state-dependent work.

Quantity	Depends on	Scheduling consequence
$\mathbf{A}$	Keys and beta in one chunk	Build chunks in parallel
$\mathbf{W}, \mathbf{U}$	$\mathbf{A}$, local keys/values/beta	Build chunks in parallel
$\mathbf{P}$	$\mathbf{W}, \mathbf{U}$, entry memory	Wait for entry memory
$\mathbf{S}_{\text{out}}$	Entry memory and actual corrections	Provides next entry memory
$\mathbf{O}$	Saved entry memory, local scores and corrections	Output chunks can run in parallel afterward

Intermediates are not interchangeable

$\mathbf{U}$ would be the correction matrix for zero entry memory. It is not the correction matrix for an arbitrary carried state. Replacing $\mathbf{P}$ with $\mathbf{U}$ silently drops predictions made from previous chunks. A test beginning with zero state and containing one chunk may fail to expose this mistake.

$\mathbf{W}$ has a key-channel width because it multiplies $\mathbf{S}_{\text{in}}$. $\mathbf{U}$ and $\mathbf{P}$ have a value-channel width because their rows are correction writes. Naming these shapes correctly helps prevent confusing the roles of `key` and `value`.

How the blocked inverse fits the starter

The supplied inverse initializes an identity and selects four 16×16 diagonal blocks. Inside each block, forward substitution subtracts earlier solved rows. Merging two neighboring blocks changes only the new lower-left block; the already-solved diagonal blocks remain intact. At the next level, combine the two 32×32 systems into the full 64×64 inverse.

The mathematical zero upper triangle must remain zero. The diagonal remains one. Check those structural invariants even when the numerical residual is small. A strict WY mask and an inclusive output mask serve different causal dependencies, so reuse the product structure from P3 while preserving the mask appropriate to each stage.

References

[1] S. Yang. *Toward More Expressive yet Scalable RNNs: DeltaNet and Its Variants*. Talk, July 15, 2025.
https://sustcsonglin.github.io/assets/pdf/deltanet_talk_slides.pdf

[2] S. Yang. *DeltaNet Explained (Part II)*. 2024.
https://sustcsonglin.github.io/blog/2024/deltanet-2/

[3] S. Yang et al. *Parallelizing Linear Transformers with the Delta Rule over Sequence Length*. 2024.
https://arxiv.org/abs/2406.06484